

Graphology-Inspired Cognitive Load Estimation System

What is this project?

This is a **Graphology-Inspired Cognitive Load Estimation System**. The core idea is to look at a handwriting image and answer: *how hard was this person's brain working when they wrote this?*

Cognitive load is not the same as stress or emotion. It specifically measures **mental effort** — the demand placed on working memory. A person copying a paragraph has low cognitive load. A person solving a math problem while writing has high cognitive load. The science behind this comes from motor cortex research: when your brain is under heavy load, the fine motor signals controlling your hand degrade in measurable ways — your strokes become shakier, your letter sizes vary, your pen angle fluctuates, you hesitate more. Graphology gives us the vocabulary to name and measure those changes.

The dataset — IAM Handwriting

The IAM Words dataset is a large publicly available collection of scanned English handwriting. Your copy has 115,320 word images organised like this:

iam_words/words/a01/a01-000u/a01-000u-07-03.png

↑ ↑ ↑ ↑ ↑

group form form line word

The filename encodes everything. a01-000u-07-03.png means: writer group a01, form 000u, line 07, word 03. There are no cognitive load labels in IAM — it was built for handwriting recognition. So we assign labels ourselves using a **line-position proxy**: lines 00–02 are labeled LOW (the writer is fresh), lines 03–05 are MEDIUM (sustained effort), lines 06 and beyond are HIGH (fatigue accumulates, working memory is taxed). This is backed by the Rosenblum et al. (2010) study which showed measurable handwriting degradation in later lines of a dictation task. Your dataset split came out as 43,316 LOW, 41,253 MEDIUM, and 30,749 HIGH samples.

The feature extraction — `utils/feature_extractor.py`

This is the scientific heart of the project. Every image passes through two stages before any ML happens.

Preprocessing (preprocess function): The image is loaded with `cv2.imread`, converted to grayscale with `cvtColor`, denoised with a 3×3 Gaussian blur to remove scanner noise, then binarised using adaptive Gaussian thresholding — this splits pixels into ink (255) and background (0). Adaptive thresholding is used instead of a fixed threshold because IAM word crops have uneven lighting across the scan. A sanity check then measures the ratio of ink pixels; if it's too high (the image is mostly noise) or too low (barely any ink), it falls back to Otsu's method which finds the optimal global threshold automatically.

After preprocessing you have two arrays for every image: gray (raw pixel intensities) and binary (clean ink mask). Fifteen features are then computed across four graphological categories.

Pressure features use OpenCV's distance transform — `cv2.distanceTransform` computes, for every ink pixel, how far it is from the nearest background pixel. The result is a map of stroke half-widths. The mean of this map is `pressure_mean` (thicker strokes = more pressure = lower load). The standard deviation is `pressure_variance` (inconsistent pressure = cognitive load marker). `ink_density` directly reads the mean darkness of ink pixels from the gray image — faint ink signals a tired or distracted hand.

Slant features use `cv2.findContours` to find every connected ink component (each letter fragment), then `cv2.fitEllipse` to fit an ellipse to each one and read off its rotation angle. The mean of all these angles is `slant_mean_angle`. The standard deviation is `slant_deviation` — the most important single feature in graphology for cognitive load, because an inconsistent pen tilt across a word reveals divided attention. `slant_skewness` uses SciPy's skew function on the angle distribution to catch asymmetric tilt patterns.

Spacing features sort all contour bounding boxes by their left edge (x coordinate) and measure the pixel gaps between adjacent boxes. `letter_spacing_mean` is the average gap — narrow spacing indicates a rushed, loaded writer. `letter_spacing_var` is the standard deviation of those gaps — erratic spacing is a strong load marker. `baseline_deviation` extracts the y-coordinate of each letter's centroid (centre of mass via `cv2.moments`) and computes the standard deviation — an unsteady baseline means the hand is drifting, which happens under cognitive overload.

Form and shape features are the richest group. `form_regularity` computes the compactness ratio $4\pi \cdot \text{area} / \text{perimeter}^2$ for each contour — perfect circles score 1.0, irregular jagged shapes score much lower. `tremor_index` divides the actual contour perimeter by the convex hull perimeter using `cv2.convexHull` — a ratio above 1.0 means the stroke is jagged rather than smooth, a direct measure of fine motor tremor. `pen_lift_fragmentation` counts how many disconnected ink components exist per unit of ink area — more fragments means more hesitations and pen lifts, which indicate uncertainty. `letter_height_var` measures the standard deviation of bounding box heights — size inconsistency under load. `pixel_entropy` computes the Shannon entropy of the full pixel intensity histogram using SciPy — a disordered, high-entropy image corresponds to chaotic writing. `zone_ratio` divides the image into horizontal thirds (upper zone for ascenders like d/h/l, middle zone for the core letter bodies, lower zone for descenders like g/p/y) and measures the ink imbalance between upper and lower zones — graphologists have long observed that zone proportions shift under cognitive stress.

The dataset preparation — `utils/prepare_dataset.py`

This script walks the entire `iam_words/words/` tree using `os.walk`, collects every PNG path, calls `extract_all_features` on each one, and writes a row to a Pandas DataFrame. Each row contains the filename, path, the auto-assigned label, the line number parsed from the filename using regex, and all 15 feature values as floats. The full DataFrame is saved as `data/dataset.csv`. The script processed 115,320 images in about 6 minutes on your machine and produced 115,318 valid rows (2 images were corrupted and skipped).

The model training — `train_model.py`

The CSV is loaded and the 15 feature columns are extracted as the feature matrix `X`. Labels are encoded with scikit-learn's `LabelEncoder` — HIGH=0, LOW=1, MEDIUM=2 (alphabetical). The data is split 80/20 into train and test sets using `train_test_split` with stratification so each label keeps its original proportion. A `StandardScaler` is fitted on the training set and applied to both sets — this normalises all features to zero mean and unit variance, which matters because features like `pen_lift_fragmentation` (values around 5–30) would otherwise dominate features like `zone_ratio` (values around 0–0.5).

Two models are trained: a `RandomForestClassifier` with 300 trees, balanced class weights, and `max_features='sqrt'`, and an `XGBClassifier` with 300 estimators, learning rate 0.08, and subsampling. Both are evaluated on the test set for accuracy and weighted F1. A 5-fold stratified cross-validation is also run. XGBoost won with 41.3% accuracy versus Random Forest's 40.4%. The model, scaler, and label encoder are all saved with `joblib.dump` into the

model/ folder alongside a model_info.json containing the metadata and three evaluation plots.

The 41% accuracy is expected and acceptable. The IAM labels are proxies, not ground truth — there is genuine overlap between the classes because not every writer degrades at the same line rate. For a viva, this is actually a strength: it shows you understand the limitation of proxy labeling and that your system still extracts meaningful signal above random chance (33%).

The Flask web app — app.py

The app loads the model, scaler, and label encoder once at startup using joblib.load. It initialises a SQLite database at data/predictions.db to store every prediction for the history table.

The /predict POST route receives the uploaded image file and the task type string. It saves the file to static/uploads/, calls extract_all_features to get the 15-feature dict and vector, scales the vector with the loaded scaler, runs MODEL.predict and MODEL.predict_proba to get the predicted class and confidence scores, then runs the task-aware logic. The task-aware logic compares the predicted load level against the expected load for the chosen task type — copying expects LOW, memory expects MEDIUM, problem solving expects HIGH. The difference in rank (using the ordering LOW=0, MEDIUM=1, HIGH=2) determines whether the status is NORMAL, SLIGHTLY_ELEVATED, or ABNORMAL. The build_insights function then checks each of 8 key features against empirical thresholds and generates a plain-English sentence for each triggered condition. The triggered value is explicitly cast to Python's native bool (the bug you just fixed — numpy booleans are not JSON serialisable). The full result dict is serialised with jsonify and returned to the browser.

The frontend — templates/index.html, static/css/style.css, static/js/main.js

The HTML defines the page structure: a sticky navbar showing model status, a pipeline progress bar with 6 steps, a two-column grid with the input panel on the left and the results panel on the right, and a history table at the bottom. The CSS uses CSS custom properties for all colours so it adapts to light and dark mode. The JS handles file selection (FileReader API to show the image preview), task button selection, and the main runAnalysis function which uses the Fetch API to POST a FormData object to /predict. While the request is in flight, a setInterval animates the pipeline bar steps. When the response arrives, renderResults builds the load circle, confidence bar, per-class probability chips, the four grouped feature cards with animated bar fills, the insights list, and then calls loadHistory to refresh the table.

Tech stack summary

Layer	Technology	Purpose
Image processing	OpenCV (cv2)	Preprocessing, contours, distance transform, moments
Numerical computing	NumPy	Array operations on pixel data
Statistics	SciPy	Shannon entropy, skewness
Machine learning	scikit-learn, XGBoost	Scaling, training, evaluation
Model persistence	joblib	Save/load model and scaler
Data handling	Pandas	Build and read the dataset CSV
Visualisation	Matplotlib, Seaborn	Confusion matrix and importance plots
Web server	Flask	REST API, file uploads, JSON responses
Database	SQLite (via Python's sqlite3)	Store prediction history
Frontend	HTML5, CSS3, Vanilla JS	UI, Fetch API, dynamic DOM updates